

4 Working with Spring Data JPA

This chapter covers

- Introducing Spring Data and its modules
- Examining the main concepts of Spring Data JPA
- Investigating the query builder mechanisms
- Examining projections, and modifying and deleting queries
- Examining Query by Example

Spring Data is an umbrella project containing many projects specific to various databases. These projects are developed in partnership with the companies creating the database technologies themselves. Spring Data's goal is to provide an abstraction for data access while retaining the underlying specifics of the various data stores.

We'll discuss the following general features provided by Spring Data:

- Integration with Spring via JavaConfig and XML configuration
- Repository and custom object-mapping abstractions
- Integration with custom repository code
- Dynamic query creation based on repository method names
- Integration with other Spring projects, such as Spring Boot

We enumerated the main Spring Data modules in chapter 2. We'll focus on Spring Data JPA here, which is largely used as an alternative for accessing databases from Java programs. It provides a layer of abstraction on top of a JPA provider (such as Hibernate), in the spirit of the Spring framework, taking control of the configuration and transactions management. We'll use it to interact with the databases in many of our examples in the following chapters, so this chapter will analyze its capabilities in depth. We will still define and manage our entities using JPA and Hibernate, but we'll provide Spring Data JPA as an alternative for interacting with them.

4.1 Introducing Spring Data JPA

Spring Data JPA provides support for interacting with JPA repositories. As you can see in figure 4.1, it is built on top of the functionality offered by the Spring Data Commons project and the JPA provider (Hibernate in our case). To review the main Spring Data modules, refer to chapter 2.

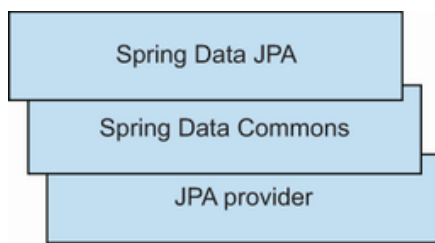


Figure 4.1 Spring Data JPA is built on top of Spring Data Commons and the JPA provider.

Throughout the book, we’ll generally interact with databases using both Hibernate JPA and Spring Data as alternatives. This chapter, along with the background provided in chapters 1–3, will help you start using the most important capabilities of Spring Data JPA. We’ll examine further Spring Data JPA features when they are needed, and we’ll look at other Spring Data projects in their dedicated chapters.

As you saw in section 2.6 when we created the “Hello World” application, Spring Data JPA can do several things to facilitate interaction with a database:

- Configure the data source bean
- Configure the entity manager factory bean
- Configure the transaction manager bean
- Manage transactions through annotations

4.2 Starting a new Spring Data JPA project

We’ll use the CaveatEmptor example application that we introduced in chapter 3 to demonstrate and analyze the capabilities of Spring Data JPA. We’ll use Spring Data JPA as a persistence framework to manage and persist the CaveatEmptor users, with Hibernate JPA as the underlying JPA provider. Spring Data JPA can execute CRUD operations and queries against a database, and it can be backed by different JPA implementations. It provides another layer of abstraction to interact with the databases.

NOTE To be able to execute the examples from the source code, you’ll first need to run the Ch04.sql script. The source code is in the `spring-datajpa` folder.

We’ll create a Spring Boot application to use Spring Data JPA. To do this, we’ll use the Spring Initializr website at <https://start.spring.io/> to create a new Spring Boot project (see figure 4.2) with the following characteristics:

- Group: `com.manning.javapersistence`
- Artifact: `springdatajpa`
- Description: Spring Data with Spring Boot

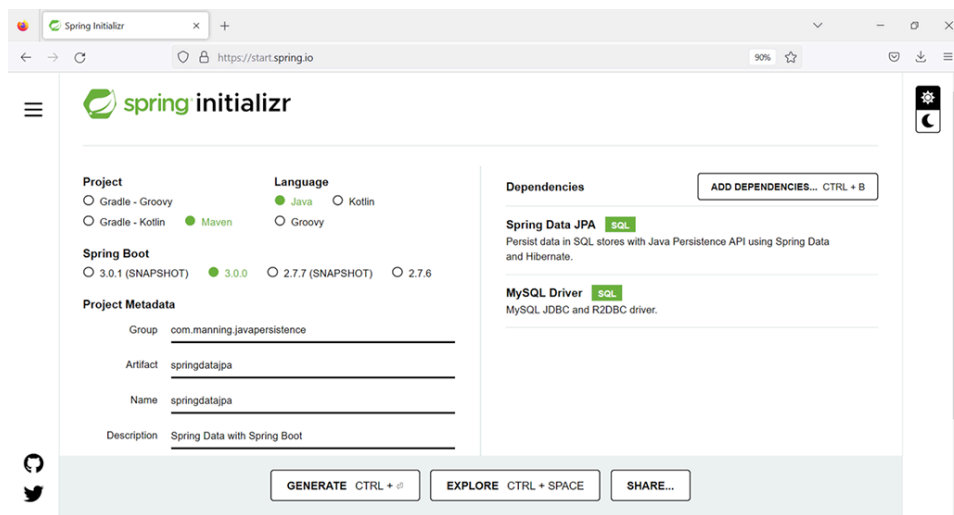


Figure 4.2 Creating a new Spring Boot project using Spring Data JPA and MySQL

We'll also add the following dependencies:

- Spring Data JPA (this will add `spring-boot-starter-data-jpa` in the Maven `pom.xml` file)
- MySQL Driver (this will add `mysql-connector-java` in the Maven `pom.xml` file)

After you click the Generate button (shown in figure 4.2), the Spring Initializr website will provide an archive to be downloaded. This archive contains a Spring Boot project that uses Spring Data JPA and MySQL. Figure 4.3 shows this project opened in the IntelliJ IDEA IDE.

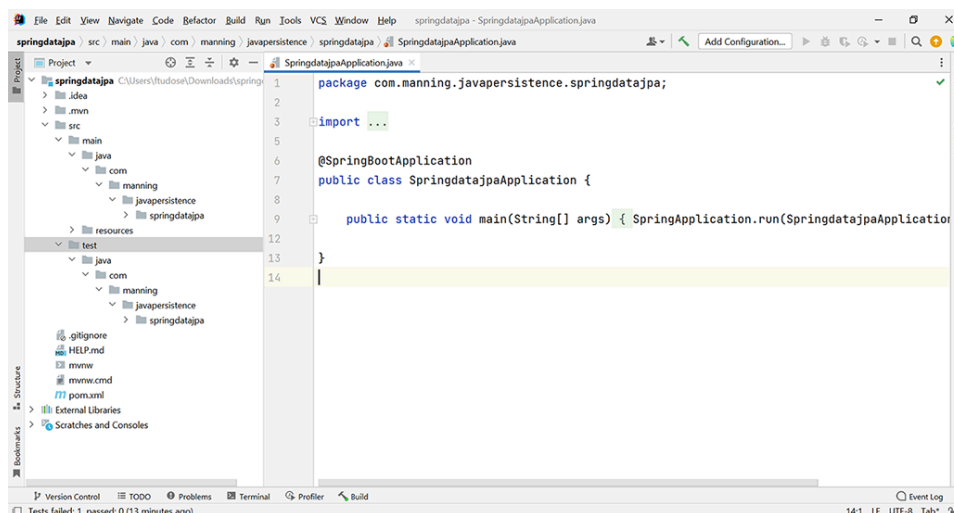


Figure 4.3 Opening the Spring Boot project that uses Spring Data JPA and MySQL

The skeleton of the project contains four files:

- `SpringDataJpaApplication` includes a skeleton `main` method.
- `SpringDataJpaApplicationTests` includes a skeleton test method.
- `application.properties` is empty at the beginning.
- `pom.xml` includes the management information needed by Maven.

As the first three files in the preceding list are standard ones, we'll take a closer look now at the pom.xml file generated by Spring Initializr.

Listing 4.1 The pom.xml Maven file

Path: Ch04/springdatajpa/pom.xml

```
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>2.7.0</version>
  <relativePath/> <!-- lookup parent from repository -->
</parent>
<groupId>com.manning.javapersistence</groupId>
<artifactId>springdatajpa</artifactId>
<version>0.0.1-SNAPSHOT</version>
<name>springdatajpa</name>
<description>Spring Data with Spring Boot</description>
<properties>
  <java.version>17</java.version>
</properties>
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>

  <dependency>
    <groupId>mysql</groupId>
    <artifactId>mysql-connector-java</artifactId>
    <scope>runtime</scope>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
  </dependency>
</dependencies>

<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
    </plugin>
  </plugins>
</build>
```

Ⓐ The parent POM is `spring-boot-starter-parent`. This parent provides default configuration, dependency, and plugin management for the Maven applications. It also inherits dependency management from its parent, `spring-boot-dependencies`.

ⓑ Indicates the `groupId`, `artifactId`, `version`, `name`, and `description` of the project, plus the Java version.

ⓒ `spring-boot-starter-data-jpa` is the starter dependency used by Spring Boot to connect to a relational database through Spring Data JPA with Hibernate. It uses Hibernate as a transitive dependency.

ⓓ `mysql-connector-java` is the JDBC driver for MySQL. It is a runtime dependency, indicating that it is not needed in the classpath for compiling, but only at runtime.

ⓔ `spring-boot-starter-test` is the Spring Boot starter dependency for testing. This dependency is needed only for the test compilation and execution phases.

ⓕ `spring-boot-maven-plugin` is a utility plugin for building and running a Spring Boot project.

4.3 First steps for configuring a Spring Data JPA project

We'll now write the class that will describe a `User` entity. The `CaveatEmptor` application has to keep track of the users interacting with it, so it is natural to start with the implementation of this class.

Listing 4.2 The `User` entity

```
Path: Ch04/springdatajpa/src/main/java/com/manning/javapersistence
➡ /springdatajpa/model/User.java
```

```
@Entity
@Table(name = "USERS")
public class User {

    @Id
    @GeneratedValue
    private Long id;

    private String username;

    private LocalDate registrationDate;

    public User() {

    }

    public User(String username) {
        this.username = username;
    }
}
```

```

    public User(String username, LocalDate registrationDate) {
        this.username = username;
        this.registrationDate = registrationDate;
    }

    public Long getId() {
        return id;
    }

    public String getUsername() {
        return username;
    }

    public void setUsername(String username) {
        this.username = username;
    }

    public LocalDate getRegistrationDate() {
        return registrationDate;
    }

    public void setRegistrationDate(LocalDate registrationDate) {
        this.registrationDate = registrationDate;
    }

    @Override
    public String toString() {
        return "User{" +
            "id=" + id +
            ", username='" + username + '\'' +
            ", registrationDate=" + registrationDate +
            '}';
    }
}

```

Ⓐ Create the `User` entity and annotate it with the `@Entity` and `@Table` annotations. We specify `USERS` as the name of the corresponding table, because the default `USER` name is reserved in most database systems.

Ⓑ Specify the `id` field as the primary key and include a getter for it. The `@GeneratedValue` annotation enables the automatic generation of `ids`. We'll look at this more in chapter 5.

Ⓒ Declare the `username` and `registrationDate` fields, together with getters and setters.

Ⓓ Declare three constructors, including a no-arguments one. Recall that JPA requires a constructor with no arguments for every persistent class. JPA uses the Java Reflection API on such a no-argument constructor to create instances.

Ⓔ Create the `toString` method to nicely display the instances of the `User` class.

Ⓕ `spring-boot-starter-test` is the Spring Boot starter dependency for testing. This dependency is needed only for the test compilation and execution phases.

Ⓖ `spring-boot-maven-plugin` is a utility plugin for building and running a Spring Boot project.

We'll also create the `UserRepository` interface.

Listing 4.3 The `UserRepository` interface

```
Path: Ch04/springdatajpa/src/main/java/com/manning/javapersistence
➡ /springdatajpa/repositories/UserRepository.java

public interface UserRepository extends CrudRepository<User, Long> {
}
```

The `UserRepository` interface extends `CrudRepository<User, Long>`. This means that it is a repository of `User` entities, which have a `Long` identifier. Remember, the `User` class has an `id` field of type `Long` annotated as `@Id`. We can directly call methods such as `save`, `findAll`, and `findById`, inherited from `CrudRepository`, and we can use them without any additional information to execute the usual operations against the database. Spring Data JPA will create a proxy class implementing the `UserRepository` interface and implement its methods.

It is worth mentioning that `CrudRepository` is a generic technology-agnostic persistence interface that we can use not only for JPA/relational databases but also for NoSQL databases. For example, we can easily change the database from MySQL to MongoDB without touching the implementation by changing the dependency from the original `spring-boot-starter-data-jpa` to `spring-boot-starter-data-mongodb`.

The next step will be to fill in the Spring Boot `application.properties` file. Spring Boot will automatically find and load the `application.properties` file from the classpath; the `src/main/resources` folder is added by Maven to the classpath.

Listing 4.4 The `application.properties` file

```
Path: Ch04/springdatajpa/src/main/resources/application.properties

spring.datasource.url=jdbc:mysql://localhost:3306/CH04_SPRINGDATAJPA
```

```

        ?serverTimezone=UTC
spring.datasource.username=root
spring.datasource.password=
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL8Dialect
spring.jpa.show-sql=true
spring.jpa.hibernate.ddl-auto=create

```

- Ⓐ The application.properties file will indicate the URL of the database.
- Ⓑ The username, and no password for access.
- Ⓒ The Hibernate dialect is MySQL8, as the database we'll interact with is MySQL Release 8.0.
- Ⓓ While executing, the SQL code is shown.
- Ⓔ Every time the program is executed, the database will be created from scratch.

We'll now write code that saves two users to the database and then tries to find them.

Listing 4.5 Persisting and finding `User` entities

```

Path: Ch04/springdatajpa/src/main/java/com/manning/javapersistence
➡ /springdatajpa/SpringDataJpaApplication.java

@SpringBootApplication
public class SpringDataJpaApplication {

    public static void main(String[] args) {
        SpringApplication.run(SpringDataJpaApplication.class, args);
    }

    @Bean
    public ApplicationRunner configure(UserRepository userRepository) {
        return env ->
        {
            User user1 = new User("beth", LocalDate.of(2020,
                ➡ Month.AUGUST, 3));
            User user2 = new User("mike",
                LocalDate.of(2020, Month.JANUARY, 18));

            userRepository.save(user1);
            userRepository.save(user2);

            userRepository.findAll().forEach(System.out::println);
        };
    }
}

```


- Ⓐ The `@SpringBootApplication` annotation, added by Spring Boot to the class containing the `main` method, will enable the Spring Boot auto-configuration mechanism and the scan on the package where the application is located, and it will allow the registration of extra beans in the context.
- Ⓑ `SpringApplication.run` will load the standalone Spring application from the `main` method. It will create an appropriate `ApplicationContext` instance and load beans.
- Ⓒ Spring Boot will run the `@Bean` annotated method, returning an `ApplicationRunner` just before `SpringApplication.run()` finishes.
- Ⓓ Create two users.
- Ⓔ Save them to the database.
- Ⓕ Retrieve them and display the information about them.

When we run this application, we'll get the following output (determined by the way the `toString()` method of the `User` class works):

```
User{id=1, username='beth', registrationDate=2020-08-03}
User{id=2, username='mike', registrationDate=2020-01-18}
```

4.4 Defining query methods with Spring Data JPA

We'll extend the `User` class by adding the fields `email`, `level`, and `active`. A user may have different levels, which will allow them to execute particular actions (such as bidding above some amount). A user may be active or may be retired (previously active in the CaveatEmptor auction system, but not anymore). This is important information that the CaveatEmptor application needs to keep about its users.

NOTE The source code we'll discuss in the rest of this chapter can be found in the `springdatajpa2` folder.

Listing 4.6 The modified `User` class

```
Path: Ch04/springdatajpa2/src/main/java/com/manning/javapersistence
➡ /springdatajpa/model/User.java
```

```
@Entity
@Table(name = "USERS")
public class User {

    @Id
    @GeneratedValue
```

```

        private Long id;

        private String username;

        private LocalDate registrationDate;

        private String email;

        private int level;

        private boolean active;

        public User() {

        }

        public User(String username) {
            this.username = username;
        }

        public User(String username, LocalDate registrationDate) {
            this.username = username;
            this.registrationDate = registrationDate;
        }

        //getters and setters
    }

```

We'll now start to add new methods to the `UserRepository` interface and use them inside newly created tests. We'll change the `UserRepository` interface to extend `JpaRepository` instead of `CrudRepository`. `JpaRepository` extends `PagingAndSortingRepository`, which, in turn, extends `CrudRepository`.

`CrudRepository` provides basic CRUD functionality, whereas `PagingAndSortingRepository` offers convenient methods that sort and paginate the records (which we'll address later in the chapter). `JpaRepository` offers JPA-related methods, such as flushing the persistence context and deleting records in a batch. Additionally, `JpaRepository` overwrites a few methods from `CrudRepository`, such as `findAll`, `findAllById`, and `saveAll` to return `List` instead of `Iterable`.

We'll also add a series of query methods to the `UserRepository` interface, as shown in the following listing

Listing 4.7 The `UserRepository` interface with new methods

Path: Ch04/springdatajpa2/src/main/java/com/manning/javapersistence
➡ /springdatajpa/repositories/UserRepository.java

```
public interface UserRepository extends JpaRepository<User, Long> {

    User findByUsername(String username);
    List<User> findAllOrderByUsernameAsc();
    List<User> findByRegistrationDateBetween(LocalDate start,
        ➡ LocalDate end);
    List<User> findByUsernameAndEmail(String username, String email);
    List<User> findByUsernameOrEmail(String username, String email);
    List<User> findByUsernameIgnoreCase(String username);
    List<User> findByLevelOrderByUsernameDesc(int level);
    List<User> findByLevelGreaterThanEqual(int level);
    List<User> findByUsernameContaining(String text);
    List<User> findByUsernameLike(String text);
    List<User> findByUsernameStartingWith(String start);
    List<User> findByUsernameEndingWith(String end);
    List<User> findByActive(boolean active);
    List<User> findByRegistrationDateIn(Collection<LocalDate> dates);
    List<User> findByRegistrationDateNotIn(Collection<LocalDate> dates);

}
```

The purpose of these query methods is to retrieve information from the database. Spring Data JPA provides a query builder mechanism that will create behavior for the repository methods based on their names. Later we'll look at modifying queries, which modify the data they find; for now, we'll focus on queries whose purpose is to find information. This query mechanism removes prefixes and suffixes such as `find...By`, `get...By`, `query...By`, `read...By`, and `count...By` from the name of the method and parses the remainder of it.

You can declare methods containing expressions as `Distinct` to set a distinct clause; declare operators as `LessThan`, `GreaterThan`, `Between`, or `Like`; or declare compound conditions with `And` or `Or`. You can apply static ordering with the `OrderBy` clause in the name of the query method, referencing a property and providing a sorting direction (`Asc` or `Desc`). You can use `IgnoreCase` for properties that support such a clause. For deleting rows, you'd have to replace `find` with `delete` in the names of the methods. Also, Spring Data JPA will look at the return type of the method. If you want to find a `User` and return it in an `Optional` container, the method return type will be `Optional<User>`. A full list of possible return types, together with detailed explanations, can be found in appendix D of the Spring Data JPA reference documentation (<http://mng.bz/o51y>).

The names of the methods need to follow the rules. If the method naming is wrong (for example, the entity property does not match in the query

method), you will get an error when the application context is loaded. Table 4.1 describes the essential keywords that Spring Data JPA supports and how each method name is transposed in JPQL. For a more comprehensive list, see appendix B at the end of this book.

Table 4.1 Essential Spring Data JPA keywords and the generated JPQL

Keyword	Example	Generated JPQL
Is, Equals	findByUsername findByUsernameIs findByUsernameEquals	. . . where e.user- name = ?1
And	findByUsernameAndRegistrationDate	. . . where e.user- name = ?1 and e.reg- istra- tiondate = ? 2
Or	findByUsernameOrRegistrationDate	. . . where e.username = ?1 or e.reg- istra- tiondate = ? 2
LessThan	findByRegistrationDateLessThan	. . . where e.reg- istra- tiondate < ? 1
LessThanEqual	findByRegistrationDateLessThanEqual	. . . where e.reg- istra- tiondate <= ?1
GreaterThan	findByRegistrationDateGreaterThan	. . . where e.reg- istra- tiondate > ? 1

GreaterThanOrEqualTo	findByRegistrationDateGreaterThanOrEqualTo	. . . where e.registrationdate >= ?1
Between	findByRegistrationDateBetween	. . . where e.registrationdate between ?1 and ?2
OrderBy	findByRegistrationDateOrderByUsernameDesc	. . . where e.registrationdate = ?1 order by e.username desc
Like	findByUsernameLike	. . . where e.username like ?1
NotLike	findByUsernameNotLike	. . . where e.username not like ?1
Before	findByRegistrationDateBefore	. . . where e.registrationdate < ?1
After	findByRegistrationDateAfter	. . . where e.registrationdate > ?1
Null, IsNull	findByRegistrationDate(Is)Null	. . . where e.registrationdate

NotNull,	findByRegistrationDate(Is)NotNull	. . .	tiondate is
IsNull			null
			where e.reg-
			istra-
			tiondate is
			not null
Not	findByUsernameNot	. . .	
			where
			e.username
			<> ?1

As a base class for future tests, we'll write a
 SpringDataJpaApplicationTests abstract class.

Listing 4.8 The SpringDataJpaApplicationTests abstract class

```

Path: Ch04/springdatajpa2/src/test/java/com/manning/javapersistence
➡ /springdatajpa/SpringDataJpaApplicationTests.java

@SpringBootTest                                     ①
@TestInstance(TestInstance.Lifecycle.PER_CLASS)     ②
abstract class SpringDataJpaApplicationTests {
    @Autowired                                       ③
    UserRepository userRepository;                 ④

    @BeforeAll                                     ⑤
    void beforeAll() {                             ⑥
        userRepository.saveAll(generateUsers());  ⑦
    }                                               ⑧

    private static List<User> generateUsers() {
        List<User> users = new ArrayList<>();

        User john = new User("john", LocalDate.of(2020, Month.APRIL, 13
        john.setEmail("john@somedomain.com");
        john.setLevel(1);
        john.setActive(true);

        //create and set a total of 10 users

        users.add(john);
        //add a total of 10 users to the list

        return users;
    }

    @AfterAll                                       ⑨

```

```

        void afterAll() {
            userRepository.deleteAll();
        }

    }
}

```

Ⓔ
Ⓔ
Ⓔ

Ⓐ The `@SpringBootTest` annotation, added by Spring Boot to the initially created class, tells Spring Boot to search the main configuration class (the `@SpringBootApplication` annotated class, for instance) and create the `ApplicationContext` to be used in the tests. Recall that the `@SpringBootApplication` annotation added by Spring Boot to the class containing the `main` method will enable the Spring Boot autoconfiguration mechanism, enable the scan on the package where the application is located, and allow the registration of extra beans in the context.

Ⓑ Using the `@TestInstance(TestInstance.Lifecycle.PER_CLASS)` annotation, we ask JUnit 5 to create a single instance of the test class and reuse it for all test methods. This will allow us to make the `@BeforeAll` and `@AfterAll` annotated methods non-static and to directly use the autowired `UserRepository` instance field inside them.

Ⓒ Autowire a `UserRepository` instance. This autowiring is possible due to the `@SpringBootApplication` annotation, which enables the scan on the package where the application is located and registers the beans in the context.

Ⓓ The `@BeforeAll` annotated method will be executed once before executing all tests from a class that extends `SpringDataJpaApplicationTests`. This method will not be static (see Ⓑ above).

Ⓔ The `@AfterAll` annotated method will be executed once, after executing all tests from a class that extends `SpringDataJpaApplicationTests`. This method will not be static (see Ⓑ above).

The next tests will extend this class and use the already populated database. To test the methods that now belong to `UserRepository`, we'll create the `FindUsersUsingQueriesTest` class and follow the same recipe for writing tests: call the repository method and verify its results.

Listing 4.9 The `FindUsersUsingQueriesTest` class

```

Path: Ch04/springdatajpa2/src/test/java/com/manning/javapersistence
➡ /springdatajpa/FindUsersUsingQueriesTest.java

public class FindUsersUsingQueriesTest extends
➡ SpringDataJpaApplicationTests {

```



```

@Test
void testFindAll() {
    List<User> users = userRepository.findAll();
    assertEquals(10, users.size());
}

@Test
void testFindUser() {
    User beth = userRepository.findByUsername("beth");
    assertEquals("beth", beth.getUsername());
}

@Test
void testFindAllByOrderByUsernameAsc() {
    List<User> users = userRepository.findAllByOrderByUsernameAsc()
    assertAll(() -> assertEquals(10, users.size()),
        () -> assertEquals("beth", users.get(0).getUsername()),
        () -> assertEquals("stephanie",
            users.get(users.size() - 1).getUsername()));
}

@Test
void testFindByRegistrationDateBetween() {
    List<User> users = userRepository.findByRegistrationDateBetween
        LocalDate.of(2020, Month.JULY, 1),
        LocalDate.of(2020, Month.DECEMBER, 31));
    assertEquals(4, users.size());
}

//more tests
}

```

4.5 Limiting query results, sorting, and paging

The `first` and `top` keywords (used equivalently) can limit the results of query methods. The `top` and `first` keywords may be followed by an optional numeric value to indicate the maximum result size to be returned. If this numeric value is missing, the result size will be 1.

`Pageable` is an interface for pagination information, but in practice we use the `PageRequest` class that implements it. This one can specify the page number, the page size, and the sorting criterion.

We'll add the methods shown in listing 4.10 to the `UserRepository` interface.

Listing 4.10 Limiting query results, sorting, and paging

Path: Ch04/springdatajpa2/src/main/java/com/manning/javapersistence
 ➡ /springdatajpa/repositories/UserRepository.java

```

User findFirstByOrderByUsernameAsc();
User findTopByOrderByRegistrationDateDesc();
Page<User> findAll(Pageable pageable);
List<User> findFirst2ByLevel(int level, Sort sort);
List<User> findByLevel(int level, Sort sort);
List<User> findByActive(boolean active, Pageable pageable);

```

Next we'll write the following tests to verify how these newly added methods work.

Listing 4.11 Testing limiting query results, sorting, and paging

```

Path: Ch04/springdatajpa2/src/test/java/com/manning/javapersistence
➡ /springdatajpa/FindUsersSortingAndPagingTest.java

public class FindUsersSortingAndPagingTest extends
    SpringDataJpaApplicationTests {

    @Test
    void testOrder() {

        User user1 = userRepository.findFirstByOrderByUsernameAsc();
        User user2 = userRepository.findTopByOrderByRegistrationDateDes
        Page<User> userPage = userRepository.findAll(PageRequest.of(1,
        List<User> users = userRepository.findFirst2ByLevel(2,
                                Sort.by("registrationDate")));

        assertAll(
            () -> assertEquals("beth", user1.getUsername()),
            () -> assertEquals("julius", user2.getUsername()),
            () -> assertEquals(2, users.size()),
            () -> assertEquals(3, userPage.getSize()),
            () -> assertEquals("beth", users.get(0).getUsername()),
            () -> assertEquals("marion", users.get(1).getUsername())
        );
    }

    @Test
    void testFindByLevel() {
        Sort.TypedSort<User> user = Sort.sort(User.class);

        List<User> users = userRepository.findByLevel(3,
            user.by(User::getRegistrationDate).descending());
        assertAll(
            () -> assertEquals(2, users.size()),
            () -> assertEquals("james", users.get(0).getUsername())
        );
    }
}

```

```

@Test
void testFindByActive() {
    List<User> users = userRepository.findByActive(true,
        PageRequest.of(1, 4, Sort.by("registrationDate")));
    assertAll(
        () -> assertEquals(4, users.size()),
        () -> assertEquals("burk", users.get(0).getUsername())
    );
}
}

```

- Ⓐ The first test will find the first user by ascending order of the user-name and the second user by descending order of the registration date.
- Ⓑ Find all users, split them into pages, and return page number 1 of size 3 (the page numbering starts with 0).
- Ⓒ Find the first two users with level 2, ordered by registration date.
- Ⓓ The second test will define a sorting criterion on the `User` class. `Sort.TypedSort` extends `Sort` and can use method handles to define the properties to sort by.
- Ⓔ Find users of level 3 and sort by registration date, descending.
- Ⓕ The third test will find the active users sorted by registration date, split them into pages, and return page number 1 of size 4 (the page numbering starts with 0).

4.6 Streaming results

Query methods returning more than one result can use standard Java interfaces such as `Iterable`, `List`, `Set`. Additionally, Spring Data supports `Streamable`, which can be used as an alternative to `Iterable` or any collection type. You can concatenate `Streamable`s and directly filter and map over the elements.

We'll add the following methods to the `UserRepository` interface.

Listing 4.12 Adding methods that return `Streamable` in the `UserRepository` interface

```

Path: Ch04/springdatajpa2/src/main/java/com/manning/javapersistence
➡ /springdatajpa/repositories/UserRepository.java

Streamable<User> findByEmailContaining(String text);
Streamable<User> findByLevel(int level);

```

We'll write the following tests to verify that these newly added methods work.

Listing 4.13 Testing methods that return `Streamable`

```
Path: Ch04/springdatajpa2/src/test/java/com/manning/javapersistence
➡ /springdatajpa/QueryResultsTest.java
```

```
@Test
void testStreamable() {
    try(Stream<User> result =
        userRepository.findByEmailContaining("someother")
            .and(userRepository.findByLevel(2))
            .stream().distinct()) {
        assertEquals(6, result.count());
    }
}
```

Ⓐ
Ⓐ
Ⓑ
Ⓒ
Ⓓ

Ⓐ The test will call the `findByEmailContaining` method, searching for emails containing “someother.”

Ⓑ The test will concatenate the resulting `Streamable` with the `Streamable` providing the users of level 2.

Ⓒ It will transform this into a stream and will keep the distinct users. The stream is given as a resource of the `try` block, so it will automatically be closed. An alternative is to explicitly call the `close()` method. Otherwise, the stream would keep the underlying connection to the database.

Ⓓ Check that the resulting stream contains six users.

4.7 The `@Query` annotation

With the `@Query` annotation, you can create a method and then write a custom query on it. When you use the `@Query` annotation, the method name does not need to follow any naming convention. The custom query can be parameterized, identifying the parameters by position or by name, and binding these names in the query with the `@Param` annotation. The `@Query` annotation can generate native queries with the `nativeQuery` flag set to `true`. You should be aware, however, that native queries can affect the portability of the application. To sort the results, you can use a `Sort` object. The properties you order by must resolve to a query property or a query alias.

Spring Data JPA supports Spring Expression Language (SpEL) expressions in queries defined using the `@Query` annotation, and Spring Data JPA supports the `entityName` variable. In a query such as `select e from`

`#{#entityName}` `e`, `entityName` is resolved based on the `@Entity` annotation. In our case, in `UserRepository` extends `JpaRepository<User, Long>`, `entityName` will resolve to `User`.

We'll add the following methods to the `UserRepository` interface.

Listing 4.14 Limiting query results, sorting, and paging

```
Path: Ch04/springdatajpa2/src/main/java/com/manning/javapersistence
➡ /springdatajpa/repositories/UserRepository.java
```

```
@Query("select count(u) from User u where u.active = ?1")
int findNumberOfUsersByActivity(boolean active);
```

```
@Query("select u from User u where u.level = :level
➡ and u.active = :active")
List<User> findByLevelAndActive(@Param("level") int level,
                               @Param("active") boolean active);
```

```
@Query(value = "SELECT COUNT(*) FROM USERS WHERE ACTIVE = ?1",
        nativeQuery = true)
int findNumberOfUsersByActivityNative(boolean active);
```

```
@Query("select u.username, LENGTH(u.email) as email_length from
        #{#entityName} u where u.username like %?1%")
List<Object[]> findByAsArrayAndSort(String text, Sort sort);
```

Ⓐ The `findNumberOfUsersByActivity` method will return the number of active users.

Ⓑ The `findByLevelAndActive` method will return the users with the `level` and `active` status given as named parameters. The `@Param` annotation will match the `:level` parameter of the query with the `level` argument of the method and the `:active` parameter of the query with the `active` argument of the method. This is especially useful when you change the order of the parameters from the signature of the method and the query is not updated.

Ⓒ The `findNumberOfUsersByActivityNative` method will return the number of users with a given `active` status. Setting the `nativeQuery` flag to `true` indicates that, unlike the previous queries, which are written with JPQL, this query is written using a native SQL that's specific to the database.

Ⓓ The `findByAsArrayAndSort` method will return a list of arrays, with each array containing the `username` and the length of the `email`, after filtering based on the `username`. The second `sort` parameter will allow you to order the result of the query based on different criteria.

We'll write the tests for these query methods, which are pretty straightforward. We'll discuss only the test written for the fourth query method, which allows a few variations of the sorting criterion.

Listing 4.15 Testing the query methods

```
Path: Ch04/springdatajpa2/src/test/java/com/manning/javapersistence
➡ /springdatajpa/QueryResultsTest.java

public class QueryResultsTest extends SpringDataJpaApplicationTests {

    // testing the first 3 query methods

    @Test
    void testFindByAsArrayAndSort() {
        List<Object[]> usersList1 =
            userRepository.findByAsArrayAndSort("ar", Sort.by("username")
        List<Object[]> usersList2 =
            userRepository.findByAsArrayAndSort("ar",
                Sort.by("email_length").descending());
        List<Object[]> usersList3 = userRepository.findByAsArrayAndSort
            ("ar", JpaSort.unsafe("LENGTH(u.email)"));
        assertAll(
            () -> assertEquals(2, usersList1.size()),
            () -> assertEquals("darren", usersList1.get(0)[0]),
            () -> assertEquals(21, usersList1.get(0)[1]),
            () -> assertEquals(2, usersList2.size()),
            () -> assertEquals("marion", usersList2.get(0)[0]),
            () -> assertEquals(26, usersList2.get(0)[1]),
            () -> assertEquals(2, usersList3.size()),
            () -> assertEquals("darren", usersList3.get(0)[0]),
            () -> assertEquals(21, usersList3.get(0)[1])
        );
    }
}
```

Ⓐ The `findByAsArrayAndSort` method will return the users whose `username` is like `%ar%`, and it will order them by `username`.

Ⓑ The `findByAsArrayAndSort` method will return the users whose `username` is like `%ar%`, and it will order them by `email_length`, descending. Note that the `email_length` alias needed to be specified inside the query to be used for ordering.

Ⓒ The `findByAsArrayAndSort` method will return the users whose `username` is like `%ar%`, and it will order them by `LENGTH(u.email)`. `JpaSort` is a class that extends `Sort`, and it can use something other than property references and aliases for sorting. The `unsafe` property handling means that the provided String is not necessarily a property or an alias but can be an arbitrary expression inside the query.

If the method naming is wrong for any of the previous methods that follow the Spring Data JPA naming conventions (for example, the entity property does not match in the query method), you will get an error when the application context is loaded. If you are using the `@Query` annotation and the query you wrote is wrong, you will get an error at runtime when executing that method. Thus, the `@Query` annotated methods are more flexible, but they also provide less safety.

4.8 Projections

Not all attributes of an entity are always needed, so we may sometimes access only some of them. For example, the frontend may reduce I/O and display only the information that will be of interest to the end user. Consequently, instead of returning instances of the root entity managed by the repository, you may want to create projections based on certain attributes of those entities. Spring Data JPA can shape return types to selectively return attributes of entities.

An interface-based projection requires the creation of an interface that declares getter methods for the properties to be included in the projection. Such an interface can also compute specific values using the `@Value` annotation and SpEL expressions. By executing queries at runtime, the execution engine creates proxy instances of the interface for each returned element and forwards the calls to the exposed methods to the target object.

We'll create a `Projection` class and add `UserSummary` as a nested interface. We'll group the projections, as they are logically connected.

Listing 4.16 Interface-based projection

```
Path: Ch04/springdatajpa2/src/main/java/com/manning/javapersistence
➡ /springdatajpa/model/Projection.java
```

```
public class Projection {

    public interface UserSummary {

        String getUsername();           ❸

        @Value("#{target.username} #{target.email}")  ❹
        String getInfo();               ❺

    }

}
```

❸ The `getUsername` method will return the `username` field.

Ⓑ The `getInfo` method is annotated with the `@Value` annotation and will return the concatenation of the `username` field, a space, and the `email` field.

How should we approach projections in practice? If we include only methods such as Ⓐ in listing 4.16, we'll create a closed projection—this is an interface whose getters all correspond to properties of the target entity. When you're working with a closed projection, the query execution can be optimized by Spring Data JPA because all the properties needed by the projection proxy are known from the beginning.

If we include methods such as Ⓑ, we create an open projection, which is more flexible. However, Spring Data JPA will not be able to optimize the query execution, because the SpEL expression is evaluated at runtime and may include any properties or combination of properties of the entity root.

In general, you should use projections when you need to provide limited information and not expose the full entity. For performance reasons, you should prefer closed projections whenever you know from the beginning which information you want to return. If you have a query that returns the full object, and you have a similar query that only returns a projection, you can use alternate naming conventions, such as naming one method `find...By` and the other method `get...By`.

A class-based projection requires the creation of a data transfer object (DTO) class that declares the properties to be included in the projection and the getter methods. Using a class-based projection is similar to using interface-based projections. However, Spring Data JPA doesn't need to create proxy classes for managing projections. Spring Data JPA will instantiate the class that declares the projection, and the properties to be included are determined by the parameter names of the constructor of the class.

The following listing adds `UsernameOnly` as a nested class of the `Projection` class.

Listing 4.17 Class-based projection

```
Path: Ch04/springdatajpa2/src/main/java/com/manning/javapersistence
➡ /springdatajpa/model/Projection.java
```

```
public class Projection {

    // . . .

    public static class UsernameOnly {                Ⓐ
        private String username;                      Ⓑ
    }
}
```



```

        public UsernameOnly(String username) {           ❸
            this.username = username;                   ❹
        }                                                ❺

        public String getUsername() {                   ❻
            return username;                             ❼
        }

    }

}

```

- ❶ The UsernameOnly class
- ❷ The username field
- ❸ The declared constructor
- ❹ The username field exposed through a getter

The methods that we'll add to the `UserRepository` interface will look like these:

```

Path: Ch04/springdatajpa2/src/main/java/com/manning/javapersistence
➡ /springdatajpa/repositories/UserRepository.java

List<Projection.UserSummary> findByRegistrationDateAfter(LocalDate date)
List<Projection.UsernameOnly> findByEmail(String username);

```

These repository methods use the same naming conventions we applied in the previous examples in this section, and they know their return types from compile time as collections of projection types. However, we can generify the return types of repository methods, which will make them dynamic. We'll add a new method to the `UserRepository` interface:

```

Path: Ch04/springdatajpa2/src/main/java/com/manning/javapersistence
➡ /springdatajpa/repositories/UserRepository.java

<T> List<T> findByEmail(String username, Class<T> type);

```

We'll write the tests for these query methods using projections.

Listing 4.18 Testing query methods using projections

```

Path: Ch04/springdatajpa2/src/test/java/com/manning/javapersistence
➡ /springdatajpa/ProjectionTest.java

```

```

public class ProjectionTest extends SpringDataJpaApplicationTests {

    @Test
    void testProjectionUsername() {

        List<Projection.UsernameOnly> users =
            userRepository.findByEmail("john@somedomain.com");

        assertAll(
            () -> assertEquals(1, users.size()),
            () -> assertEquals("john", users.get(0).getUsername())
        );
    }

    @Test
    void testProjectionUserSummary() {
        List<Projection.UserSummary> users =
            userRepository.findByRegistrationDateAfter(
                LocalDate.of(2021, Month.FEBRUARY, 1));

        assertAll(
            () -> assertEquals(1, users.size()),
            () -> assertEquals("julius", users.get(0).getUsername()),
            () -> assertEquals("julius julius@someotherdomain.com",
                                users.get(0).getInfo())
        );
    }

    @Test
    void testDynamicProjection() {
        List<Projection.UsernameOnly> usernames =
            userRepository.findByEmail("mike@somedomain.com",
                Projection.UsernameOnly.class);
        List<User> users =
            ➡ userRepository.findByEmail("mike@somedomain.com",
                User.class);

        assertAll(
            () -> assertEquals(1, usernames.size()),
            () -> assertEquals("mike", usernames.get(0).getUsername()),
            () -> assertEquals(1, users.size()),
            () -> assertEquals("mike", users.get(0).getUsername())
        );
    }
}

```

Ⓐ The `findByEmail` method will return a list of `Projection.UsernameOnly` instances.

Ⓑ Verify the assertions.

- Ⓒ The `findByRegistrationDateAfter` method will return a list of `Projection .UserSummary` instances.
- Ⓓ Verify the assertions.
- Ⓔ This `findByEmail` method provides a dynamic projection. It will return a list of `Projection.UsernameOnly` instances.
- Ⓕ This `findByEmail` method may also return a list of `User` instances, depending on the class that it is generified by.
- Ⓖ Verify the assertions.

4.9 Modifying queries

You can define modifying methods with the `@Modifying` annotation. For example, `INSERT`, `UPDATE`, and `DELETE` queries, or DDL statements, modify the content of the database. The `@Query` annotation will have the modifying query as an argument, and it may need binding parameters. Such a method must also be annotated with `@Transactional` or be run from a programmatically managed transaction. Modifying queries have the advantage of clearly emphasizing which column they address, and they may include conditions, so they can make the code clearer, compared to persisting or deleting the whole object. Also, changing a limited number of columns in the database will execute more quickly.

Spring Data JPA can also generate delete queries based on method names. The mechanism works much like the examples in table 4.1, but replacing the `find` keyword with `delete`.

We'll add the following methods to the `UserRepository` interface.

Listing 4.19 Adding modifying methods to the `UserRepository` interface

```
Path: Ch04/springdatajpa2/src/main/java/com/manning/javapersistence
➡ /springdatajpa/repositories/UserRepository.java

@Transactional
@Query("update User u set u.level = ?2 where u.level = ?1")
int updateLevel(int oldLevel, int newLevel);

@Transactional
int deleteByLevel(int level);

@Transactional
@Modifying
```

Ⓐ
Ⓐ
Ⓐ
Ⓐ
Ⓑ
Ⓑ
Ⓒ
Ⓒ

```
@Query("delete from User u where u.level = ?1")
int deleteBulkByLevel(int level);
```

©
©

Ⓐ The `updateLevel` method will change `level` for users with the `oldLevel` parameter and set it to `newLevel`, as the argument of the `@Query` annotation indicates. The method is also annotated with `@Modifying` and `@Transactional`.

Ⓑ The `deleteByLevel` method will generate a query based on the method name; it will remove all users with the `level` given as a parameter. The method is annotated with `@Transactional`. `@Modifying` isn't necessary in this case, since the query is generated by the framework.

Ⓒ The `deleteBulkByLevel` method will remove all users with the `level` given as a parameter, as the argument of the `@Query` annotation indicates. The method is also annotated with `@Modifying` and `@Transactional`.

What is the difference between the `deleteByLevel` and `deleteBulkByLevel` methods? The first one runs a query, and it will then remove the returned instances one by one. If there are callback methods that control the lifecycle of each instance (for example, a method to be run when a user is removed), they will be executed. The second method will remove the users in bulk, executing a single JPQL query. No `User` instance (not even the ones that are already loaded in memory) will execute lifecycle callback methods.

We can now write tests for the modifying methods.

Listing 4.20 Testing modifying methods

```
Path: Ch04/springdatajpa2/src/test/java/com/manning/javapersistence
➡ /springdatajpa/ModifyQueryTest.java

@Test
void testModifyLevel() {
    int updated = userRepository.updateLevel(5, 4);
    List<User> users = userRepository.findByLevel(4, Sort.by("username")

    assertEquals(1, updated),
    assertEquals(3, users.size()),
    assertEquals("katie", users.get(1).getUsername())
);
}
```

We'll also write tests for the deleting methods.

```

Path: Ch04/springdatajpa2/src/test/java/com/manning/javapersistence
➡ /springdatajpa/DeleteQueryTest.java

@Test
void testDeleteByLevel() {
    int deleted = userRepository.deleteByLevel(2);
    List<User> users = userRepository.findByLevel(2, Sort.by("username"));
    assertEquals(0, users.size());
}

@Test
void testDeleteBulkByLevel() {
    int deleted = userRepository.deleteBulkByLevel(2);
    List<User> users = userRepository.findByLevel(2, Sort.by("username"));
    assertEquals(0, users.size());
}

```

4.10 Query by Example

Query by Example (QBE) is a querying technique that does not require you to write classical queries to include entities and properties. It allows dynamic query creation and consists of three pieces: a probe, an `ExampleMatcher`, and an `Example`.

The probe is a domain object with already-set properties. The `ExampleMatcher` provides the rules for matching particular properties. An `Example` puts the probe and the `ExampleMatcher` together and generates the query. Multiple `Examples` may reuse a single `ExampleMatcher`.

These are the most appropriate use cases for QBE:

- When you are decoupling the code from the underlying data store API.
- When there are frequent changes to the internal structure of the domain objects, and they aren't propagated to the existing queries.
- When you are building a set of static or dynamic constraints to query the repository.

QBE has a couple of limitations:

- It only supports starting/ending/containing regex matching for String properties, and exact matching for other types.
- It does not support nested or grouped property constraints, such as `username = ?0 or (username = ?1 and email = ?2)`.

We won't add any more methods to the `UserRepository` interface. We'll only write tests to build the probe, the `ExampleMatchers`, and the `Examples`.

Listing 4.22 Query By Example tests

```
Path: Ch04/springdatajpa2/src/test/java/com/manning/javapersistence
➡ /springdatajpa/QueryByExampleTest.java

public class QueryByExampleTest extends SpringDataJpaApplicationTests {

    @Test
    void testEmailWithQueryByExample() {
        User user = new User();
        user.setEmail("@someotherdomain.com");

        ExampleMatcher matcher = ExampleMatcher.matching()
            .withIgnorePaths("level", "active")
            .withMatcher("email", match -> match.endsWith());

        Example<User> example = Example.of(user, matcher);

        List<User> users = userRepository.findAll(example);

        assertEquals(4, users.size());
    }

    @Test
    void testUsernameWithQueryByExample() {
        User user = new User();
        user.setUsername("J");

        ExampleMatcher matcher = ExampleMatcher.matching()
            .withIgnorePaths("level", "active")
            .withStringMatcher(ExampleMatcher.StringMatcher.STARTIN)
            .withIgnoreCase();

        Example<User> example = Example.of(user, matcher);

        List<User> users = userRepository.findAll(example);

        assertEquals(3, users.size());
    }
}
```

Ⓐ Initialize a `User` instance and set up an `email` for it. This will represent the probe.

Ⓑ Create the `ExampleMatcher` with the help of the builder pattern. Any `null` reference property will be ignored by the matcher. However, we need to explicitly ignore the `level` and `active` properties, which are

primitives. If they were not ignored, they would be included in the matcher with their default values (0 for `level` and `false` for `active`) and would change the generated query. We'll configure the matcher condition so that the `email` property will end with a given string.

Ⓒ Create an `Example` that puts the probe and `ExampleMatcher` together and generates the query. The query will search for users that have an `email` property ending with the string defining the `email` of the probe.

Ⓓ Execute the query to find all users matching the probe.

Ⓔ Verify that there are four users of this kind.

Ⓕ Initialize a `User` instance and set up a `name` for it. This will represent the second probe.

Ⓖ Create the `ExampleMatcher` with the help of the builder pattern. Any `null` reference property will be ignored by the matcher. Again, we need to explicitly ignore the `level` and `active` properties, which are primitives. We configure the matcher condition so that the match will be made on starting strings for the configured properties (the `username` property from the probe, in our case).

Ⓗ Create an `Example` that puts the probe and the `ExampleMatcher` together and generates the query. The query will search for users having a `username` property that starts with the string defining the `username` of the probe.

Ⓙ Execute the query to find all users matching the probe.

Ⓚ Verify that there are six users of this kind.

To emphasize the importance of ignoring the default primitive properties, we'll compare the generated queries with and without the calls to the `withIgnorePaths ("level", "active")` methods. For the first test, this is the query generated with the call to the `withIgnorePaths ("level", "active")` method:

```
select user0_.id as id1_0_, user0_.active as active2_0_, user0_.email as email3_0_, user0_.level as level4_0_, user0_.registration_date as registra5_0_, user0_.username as username6_0_ from users user0_ where user0_.email like ? escape ?
```

This is the query generated without the call to the `withIgnorePaths ("level", "active")` method:

```

select user0_.id as id1_0_, user0_.active as active2_0_, user0_.email a
➤ email3_0_, user0_.level as level4_0_, user0_.registration_date as
➤ registra5_0_, user0_.username as username6_0_ from users user0_ where
➤ user0_.active=? and (user0_.email like ? escape ?) and user0_.level=0

```

For the second test, this is the query generated with the call to the `withIgnorePaths("level", "active")` method:

```

select user0_.id as id1_0_, user0_.active as active2_0_, user0_.email a
➤ email3_0_, user0_.level as level4_0_, user0_.registration_date as
➤ registra5_0_, user0_.username as username6_0_ from users user0_ where
➤ lower(user0_.username) like ? escape ?

```

This is the query generated without the call to the `withIgnorePaths("level", "active")` method:

```

select user0_.id as id1_0_, user0_.active as active2_0_, user0_.email a
➤ email3_0_, user0_.level as level4_0_, user0_.registration_date as
➤ registra5_0_, user0_.username as username6_0_ from users user0_ where
➤ user0_.active=? and user0_.level=0 and (lower(user0_.username) like ?
➤ escape ?)

```

Note the conditions added on primitive properties when the `withIgnorePaths("level", "active")` method was removed:

```

user0_.active=? and user0_.level=0

```

This will change the query result.

Summary

- You can create and configure a Spring Data JPA project using Spring Boot.
- You can define and use a series of query methods to access repositories by using the Spring Data JPA query builder mechanisms.
- Spring Data JPA provides capabilities for limiting query results, sorting, paging, and streaming the results.
- You can use the `@Query` annotation to define both non-native and native custom queries.
- You can implement projections to shape the return types and selectively return attributes of entities, and you can create and use modifying queries to update and delete entities.
- The Query by Example (QBE) querying technique allows for dynamic query creation and consists of three pieces: a probe, an `ExampleMatcher`, and an `Example`.

